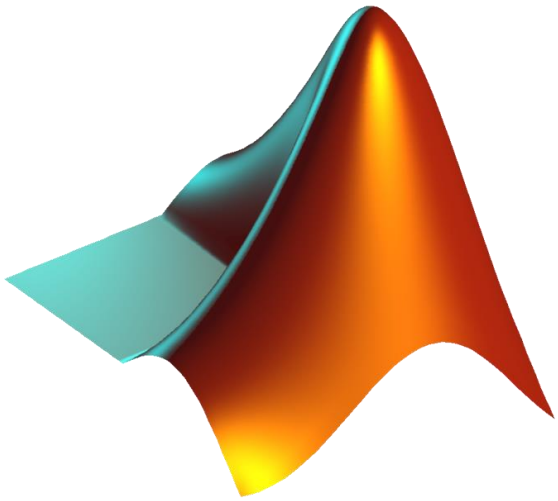
-

**Instituto Superior de Engenharia de Coimbra**
Departamento de Engenharia Informática e de Sistemas
Licenciatura em Engenharia Informática

# Atividade 05

Métodos Numéricos para Derivação e Integração



**Docente**
Arménio Correia

**Discentes**
Diogo Gomes nº 2021137427
Ricardo Rodrigues nº 2022147797
Rodrigo Gonçalves nº 2022145597

**Coimbra, 2023**

Análise Matemática II
2022/2023

# ÍNDICE

**1. Introdução** ..... 3
**2. Métodos Numéricos para Derivação** ..... 4
**2.1. Fórmulas** ..... 4
**2.1.1. Diferenças finitas em 2 pontos** ..... 4
**2.1.2. Diferenças finitas em 3 pontos** ..... 4
**2.2. Funções** ..... 5
**2.2.1. Diferenças finitas em 2 pontos** ..... 5
..... 5
**2.2.2 Diferenças finitas em 3 pontos** ..... 6
**3. Derivação simbólica no MatLab** ..... 8
**4. Métodos Numéricos para Integração** ..... 9
**4.1. Fórmulas** ..... 9
**4.1.1. Regra dos Trapézios** ..... 9
**4.1.2. Regra de Simpson** ..... 9
**4.2. Funções** ..... 10
**4.2.1. Regra dos Trapézios** ..... 10
**4.2.2. Regra de Simpson** ..... 10
**5. Exemplos de Aplicação** ..... 11
**5.1 . Derivação Numérica** ..... 11
**6. Conclusão** ..... 13
**7. Webgrafia** ..... 13

# 1. Introdução

Este trabalho surge no âmbito da unidade curricular **Análise Matemática II** e consiste na implementação em MatLab de métodos de derivação e integração numérica.

Será criada uma GUI capaz de resolver problemas de derivação e integração numérica tendo por base as funções implementadas nas duas primeiras partes do projeto, nomeadamente: Diferenças finitas em 2 e 3 pontos (Progressivas, Regressivas e Centradas), 2ª derivada e também regra dos Trapézios e regra de Simpson.

## *Enunciado:*

**Parte01**: Implemente em Matlab funções para as fórmulas de diferenças finitas seguintes
Frequentemente somos confrontados com a necessidade de determinar valores da derivada de uma função num conjunto de pontos, conhecendo apenas o valor da função nesses pontos.

**Questão:** Como aproximar o valor da derivada num ponto?
**Resposta:** Aplicar uma das fórmulas de diferenças finitas: progressivas, regressivas ou centradas.

Seja $f$ uma função definida em $[a, b]$ e suficientemente regular, conhecida num conjunto de pontos da partição uniforme
$a = x_0 < x1 < \cdots < x_n = b$

Fórmulas de diferenças finitas em 2 pontos:

Progressivas » $f'(x_k) := \frac{f(x_{k+1}) - f(x_k)}{h}$

Regressivas » $f'(x_k) := \frac{f(x_k) - f(x_{k-1})}{h}$

Fórmulas de diferenças finitas em 3 pontos:

Progressivas » $f'(x_k) := \frac{-3f(x_k) + 4f(x_{k+1}) - f(x_{k+2})}{2h}$

Regressivas » $f'(x_k) := \frac{f(x_{k-2}) - 4f(x_{k-1}) + 3f(x_k)}{2h}$

Centradas » $f'(x_k) := \frac{f(x_{k+1}) - f(x_{k-1})}{2h}$

2ª derivada » $f''(x_k) := \frac{f(x_{k+1}) - 2f(x_k) + f(x_{k-1})}{h^2}$

**Parte02**: Implemente as regras dos trapézios e Simpson seguintes

Integração Numérica
$\int_a^b f(x)\mathrm{d}x \approx$

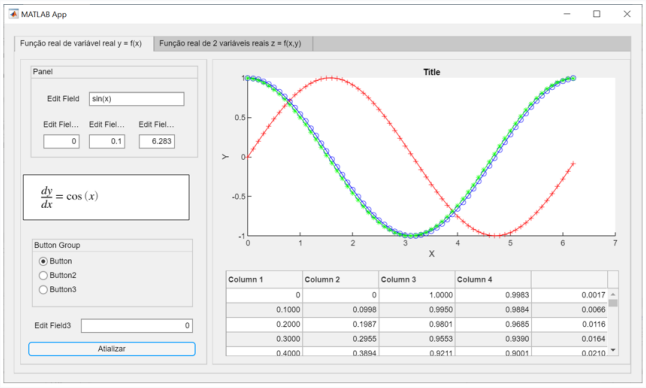
Regra dos Trapézios
$I_T(f) = \frac{h}{2}[f(x_0) + 2f(x_1) + \cdots + 2f(x_{n-1}) + f(x_n)]$
$|E_T| \leq \frac{b-a}{12} h^2 M_2, \quad M_2 = \max_{x \in [a,b]} |f''(x)|$

Regra de Simpson
$I_S(f) = \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)]$
$|E_S| \leq \frac{b-a}{180} h^4 M_4, \quad M_4 = \max_{x \in [a,b]} |f^{(4)}(x)|$

| Trapezios rule Algorithm | Simpson's rule Algorithm |
|---|---|
| Input parameters: *f, a, b* e *n*<br>Output parameter: *T*<br><br>*h* := (*b*-*a*)/*n*;<br>*x* := *a*;<br>*s* := 0;<br>**For** *i* by 1 to n-1 do<br>  x := x + h;<br>  s := s + f(x);<br>**End for**<br>*T* := *h*/2(*f*(*a*)+2*s*+*f*(*b*)) | Input parameters: *f, a, b* e *n*<br>Output parameter: *out_S*<br><br>*h* := (*b*-*a*)/*n*;<br>*x* := *a*;<br>*s* := 0;<br>**For** *i* by 1 to n-1 do<br>  x := x + h;<br>  **If** *i* is even<br>  **Then** s := s + 2f(x);<br>  **Else** s := s + 4f(x);<br>**End for**<br>*out_S* := *h*/3(*f*(*a*)+*s*+*f*(*b*)) |

**Parte03**:
Avaliar se uma função real de duas variáveis reais é harmónica.
Outras operações e avaliações...

**Parte04**: Máquina Cálculo Diferencial e Integral
Com base na máquina representada na figura seguinte, que foi objeto de trabalho nas aulas práticas, complete a máquina e amplie as suas potencialidades para o cálculo diferencial e integral de funções reais de uma variável real e de duas variáveis reais.




**Parte05**: Relatório da Atividade05Trabalho
Elabore um relatório da atividade e trabalho 05.

## 2. Métodos Numéricos para Derivação

### 2.1. Fórmulas

O valor de ***h*** corresponde ao passo de discretização para o intervalo [a, b] e será utilizado em todos os métodos numéricos implementados.

**Fórmula utilizada para o cálculo de *h*** : $$h = \frac{b-a}{n}$$

#### 2.1.1. Diferenças finitas em 2 pontos

Progressivas » $$f'(x_k) := \frac{f(x_{k+1}) - f(x_k)}{h}$$

Regressivas » $$f'(x_k) := \frac{f(x_k) - f(x_{k-1})}{h}$$

#### 2.1.2. Diferenças finitas em 3 pontos

Progressivas » $$f'(x_k) := \frac{-3f(x_k) + 4f(x_{k+1}) - f(x_{k+2})}{2h}$$

Regressivas » $$f'(x_k) := \frac{f(x_{k-2}) - 4f(x_{k-1}) + 3f(x_k)}{2h}$$

Centradas » $$f'(x_k) := \frac{f(x_{k+1}) - f(x_{k-1})}{2h}$$

2ª derivada » $$f''(x_k) := \frac{f(x_{k+1}) - 2f(x_k) + f(x_{k-1})}{h^2}$$

## 2.2. Funções

### 2.2.1. Diferenças finitas em 2 pontos

Progressivas:

```
% Formúla das Diferenças Progressivas (2 pontos)
%
% f'(x(i))=(f(x(i+1))-f(x(i))/h
%
% INPUT:  f - função
%         [a, b] - intervalo de derivação
%         h - passo da discretização
%         y - imagens x vs y
%
% OUTPUT: [x, y] - malha de pontos
%         dydx - derivada de f

%  08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797


function [x,y,dydx] = DF2_P(f,a,b,h,y)

x=a:h:b;                                % Alocação de memória

n=length(x);                            % Número de pontos (tamanho do vetor de abcissas)

if nargin==4
    y=f(x);                             % y é a função f(x)
end

dydx=zeros(1,n);                        % Alocação de memória

for i=1:n-1
    dydx(i)=(y(i+1)-y(i))/h;            % Derivada (aproximada) de f no ponto atual
end

dydx(n)=(y(n)-y(n-1))/h;                % Derivada (aproximada) de f no ponto atual
```

Regressivas:

```
% Formúla das Diferenças Regressivas (2 pontos)
%
% f'(x(i))=(f(x(i))-f(x(i-1))/h
%
% INPUT:  f - função
%         [a, b] - intervalo de derivação
%         h - passo da discretização
%         y - imagens x vs y
%
% OUTPUT: [x, y] - malha de pontos
%         dydx - derivada de f
%
%  08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797


function [x,y,dydx] = DF2_R(f,a,b,h,y)

x=a:h:b;                                % Alocação de memória

n=length(x);                            % Número de pontos (tamanho do vetor de abcissas)

if nargin==4
    y=f(x);                             % y é a função f(x)
end

dydx=zeros(1,n);                        % Alocação de memória

dydx(1)=(y(2)-y(1))/h;                  % Derivada (aproximada) de f no ponto atual

for i=2:n
    dydx(i)=(y(i)-y(i-1))/h;            % Derivada (aproximada) de f no ponto atual
end
```

### 2.2.2 Diferenças finitas em 3 pontos

Progressivas:

```
% Formúla das Diferenças Progressivas (3 pontos)
%
% f'(x(i))=(-3*f(x(i)) + 4*f(x(i+1)) + f(x(i+2))/(2*h)
%
% INPUT:  f - função
%         [a, b] - intervalo de derivação
%         h - passo da discretização
%         y - imagens x vs y
%
% OUTPUT: [x, y] - malha de pontos
%         dydx - derivada de f
%
% 08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797

function [x,y,dydx] = DF3_P(f,a,b,h,y)

x=a:h:b;                                            % Alocação de memória

n=length(x);                                        % Número de pontos (tamanho do vetor de abcissas)

if nargin==4
    y=f(x);                                         % y é a função f(x)
end

dydx=zeros(1,n);                                    % Alocação de memória

for i=1:n-2
    dydx(i)=(-3*y(i) + 4*y(i+1) - y(i+2))/(2*h);    % Derivada (aproximada) de f no ponto atual
end

dydx(n-1)=(y(n-3) - 4*y(n-2) + 3*y(n-1))/(2*h);     % Derivada (aproximada) de f no ponto atual
dydx(n)=(y(n-2) - 4*y(n-1) + 3*y(n))/(2*h);         % Derivada (aproximada) de f no ponto atual
```

Regressivas:

```
% Derivação Numérica - Fórmula das Diferenças Regressivas (3 pontos)
%
% f'(x(i))=(f(x(i-2)) - 4*f(x(i-1)) + 3*f(x(i)))/(2*h)
%
% INPUT:  f - função
%         [a, b] - intervalo de derivação
%         h - passo da discretização
%         y - imagens x vs y
%
% OUTPUT: [x, y] - malha de pontos
%         dydx - derivada de f
%
%  08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797

function [x,y,dydx] = DF3_R(f,a,b,h,y)

x=a:h:b;                                            % Alocação de memória

n=length(x);                                        % Número de pontos (tamanho do vetor de abcissas)

if nargin==4
    y=f(x);                                         % y é a função f(x)
end

dydx=zeros(1,n);                                    % Alocação de memória

dydx(1)=(-3*y(1) + 4*y(2) - y(3))/(2*h);            % Derivada (aproximada) de f no ponto atual
dydx(2)=(-3*y(2) + 4*y(3) - y(4))/(2*h);            % Derivada (aproximada) de f no ponto atual

for i=3:n
    dydx(i)=(y(i-2) - 4*y(i-1) + 3*y(i))/(2*h);     % Derivada (aproximada) de f no ponto atual
end
```

Centradas:

```
% Formúla das Diferenças Centradas (3 pontos)
%
% f'(x(i))=(f(x(i+1))-f(x(i-1))/(2*h)
%
% INPUT:  f - função
%         [a, b] - intervalo de derivação
%         h - passo da discretização
%         y - imagens x vs y
%
% OUTPUT: [x, y] - malha de pontos
%         dydx - derivada de f
%
% 08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797

function [x,y,dydx] = DF3_C(f,a,b,h,y)

x=a:h:b;                                        % Alocação de memória

n=length(x);                                    % Número de pontos (tamanho do vetor de abcissas)

if nargin==4
    y=f(x);                                     % y é a função f(x)
end

dydx=zeros(1,n);                                % Alocação de memória

dydx(1)=(-3*y(1) + 4*y(2) - y(3))/(2*h);        % Derivada (aproximada) de f no ponto atual

for i=2:n-1
    dydx(i)=(y(i+1)-y(i-1))/(2*h);              % Derivada (aproximada) de f no ponto atual
end

dydx(n)=(y(n-2) - 4*y(n-1) + 3*y(n))/(2*h);     % Derivada (aproximada) de f no ponto atual
```

## 3. Derivação simbólica no MatLab

A Symbolic Math Toolbox disponibiliza o comando ***diff*** que permite calcular derivadas de diferentes ordens.

No âmbito do nosso projeto, apenas iremos calcular derivadas de primeira e segunda ordem.

```
try
    dydxSym = diff(f(x));
    sedydxSym = diff(f(x),2);  %calcula s segunda derivada
    g =@(x) eval(vectorize(char(dydxSym)));
    o =@(x) eval(vectorize(char(sedydxSym)));
```

# 4. Métodos Numéricos para Integração

## 4.1. Fórmulas

O valor de ***h*** corresponde ao passo de discretização para o intervalo [a, b] e será utilizado em todos os métodos numéricos implementados.

**Fórmula utilizada para o cálculo de *h*** :

$$h = \frac{b-a}{n}$$

### 4.1.1. Regra dos Trapézios

$$I_T(f) = \frac{h}{2}[f(x_0) + 2f(x_1) + \cdots + 2f(x_{n-1}) + f(x_n)]$$

$$|E_T| \leq \frac{b-a}{12} h^2 M_2, \quad M_2 = \max_{x \in [a,b]} |f''(x)|$$

### 4.1.2. Regra de Simpson

$$I_S(f) = \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)]$$

$$|E_S| \leq \frac{b-a}{180} h^4 M_4, \quad M_4 = \max_{x \in [a,b]} |f^{(4)}(x)|$$

## 4.2. Funções

### 4.2.1. Regra dos Trapézios

```
% Formula da Regra dos Trapezios
%
% INPUT:  f - funcao integranda
%         [a, b] - intervalo de integracao
%         n - numero de subintervalos
%
% OUTPUT: area - Valor da area calculada pela Regra dos Trapezios
%
% 08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797

function area = Trapezios(f,a,b,n)

h=(b-a)/n;                      % Valor de cada subintervalo (passo)
x=a;                            % 'x' recebe o valor de 'a' (primeira abcissa)
s=0;                            % Inicializacao da variavel 's' a 0

for i=1:n-1
    x=x+h;                      % Ao valor de 'x' e somado o passo ('h')
    s=s+f(x);                   % Ao valor de 's' e somado o valor da funcao
end
area = (h/2)*(f(a)+2*s+f(b));
```

### 4.2.2. Regra de Simpson

```
% Formula da Regra de Simpson
%
% INPUT:  f - funcao integranda
%         [a, b] - intervalo de integracao
%         n - numero de subintervalos
%
% OUTPUT: area - Valor da area calculada pela Regra de Simpson
%
% 08/06/23
% Diogo Gomes - 2021137427
% Rodrigo Gonçalves - 2022145597
% Ricardo Rodrigues - 2022147797

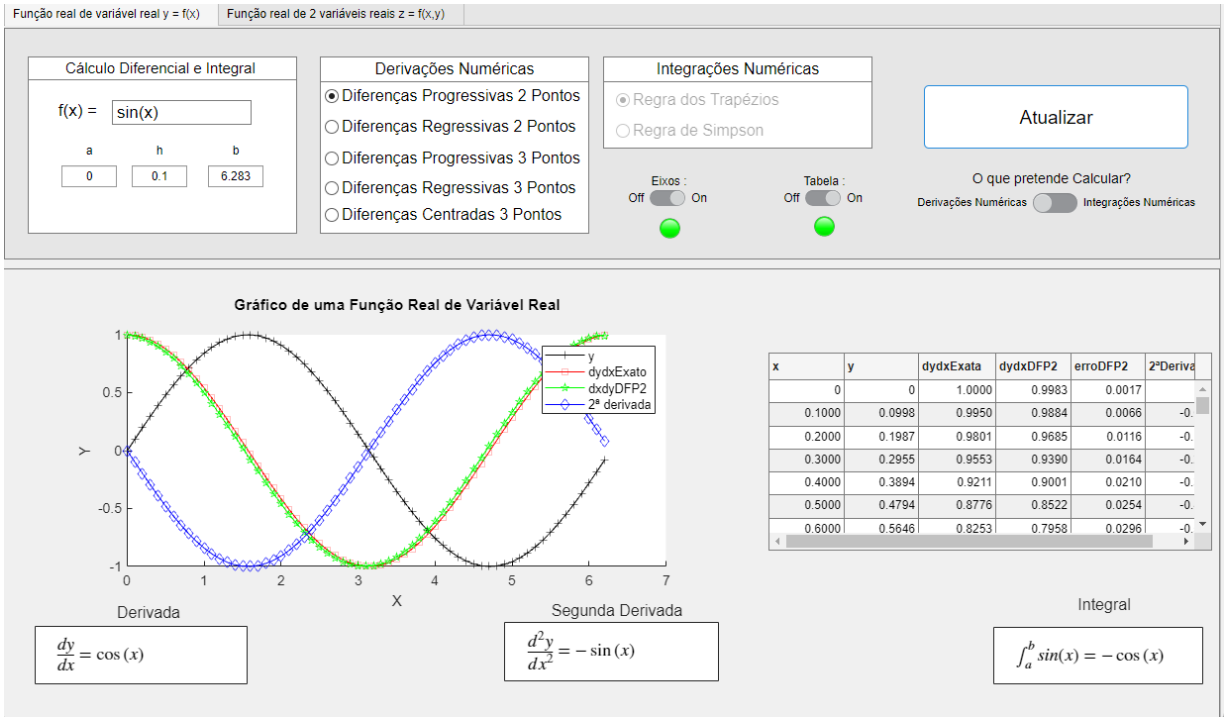
function area = Simpson(f,a,b,n)

h=(b-a)/n;                      % Valor de cada subintervalo (passo)
x=a;                            % 'x' recebe o valor de 'a' (primeira abcissa)
s=0;                            % Inicializacao da variavel 's' a 0

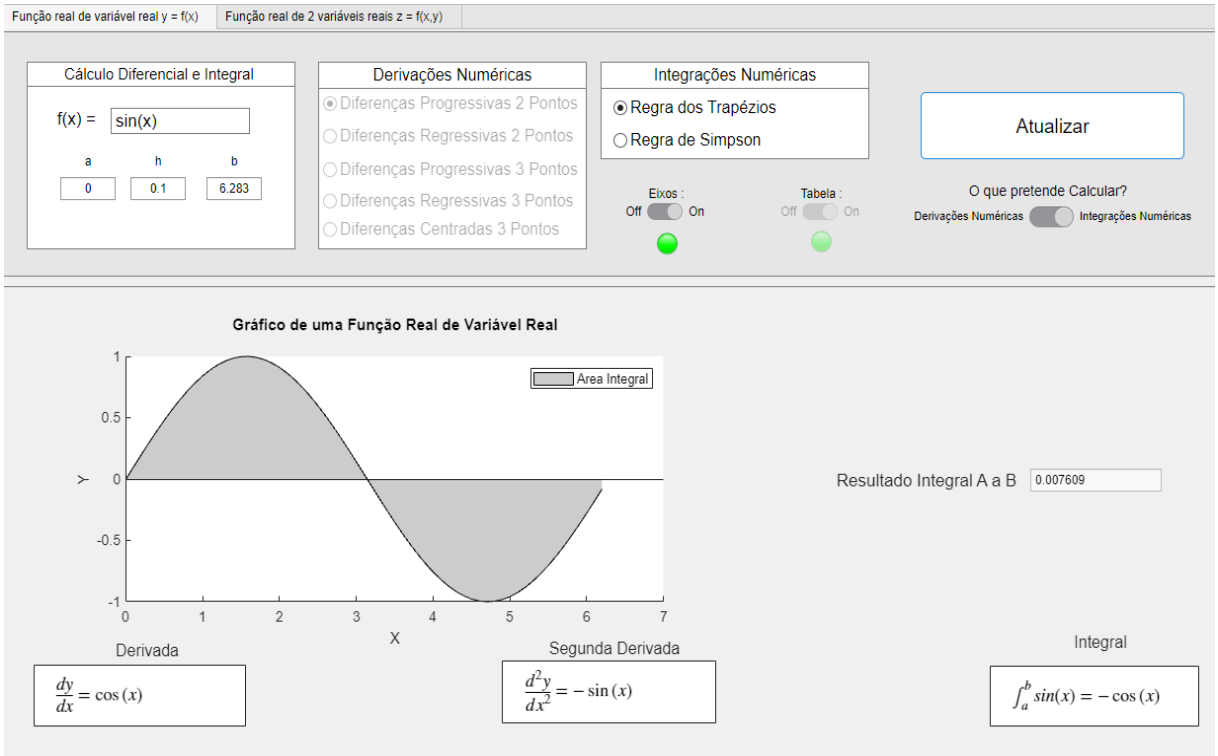
for i=1:n-1
    x=x+h;                      % Ao valor de 'x' e somado o passo ('h')
    if mod(i,2) == 0            % Se i for par
        s=s+2*f(x);             % Ao valor de 's' e somado 2 vezes o valor da funcao
    else
        s=s+4*f(x);             % Ao valor de 's' e somado 4 vezes o valor da funcao
    end
end
area = (h/3)*(f(a)+s+f(b));
```

# 5. Exemplos de Aplicação

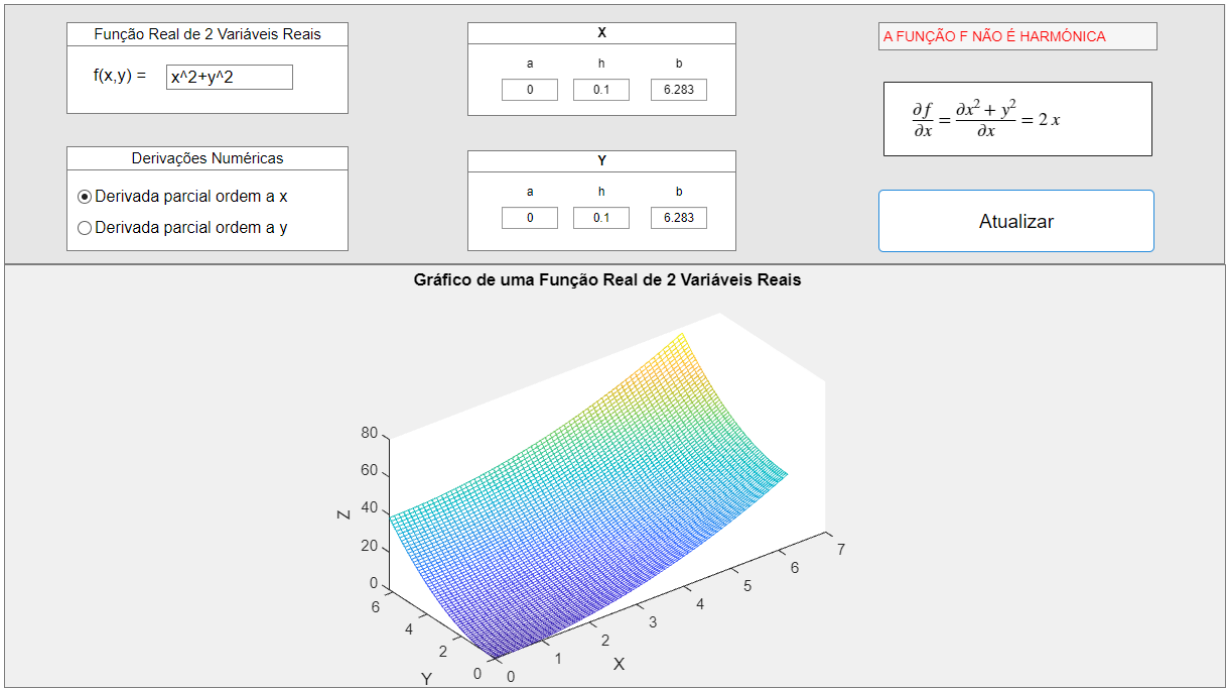
## 5.1. Derivação Numérica

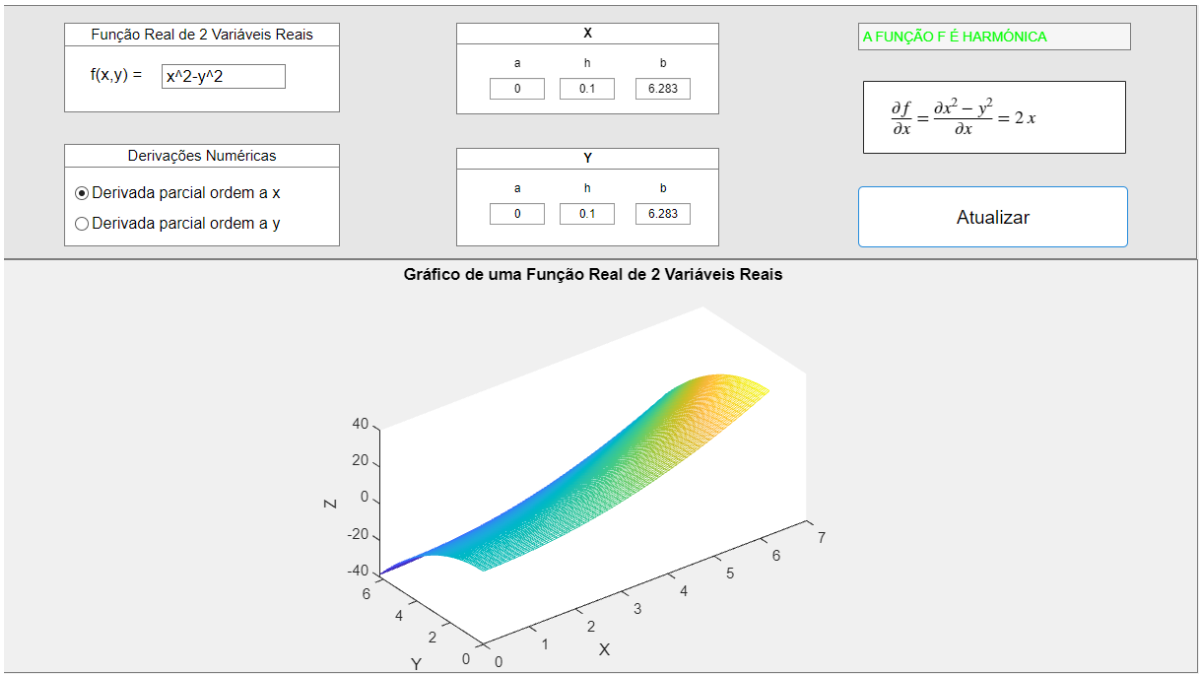



## 5.2. Integração Numérica



## 5.3. Função Real de 2 variáveis reais



## 6. Conclusão

A realização deste trabalho contribuiu para a melhor consolidação dos conhecimentos adquiridos nas aulas e evoluiu a nossa habilidade na programação com MatLab.

Além disso, conseguimos desenvolver nosso raciocínio matemático tanto o nosso espírito de equipa.

## 7. Webgrafia


**https://www.mathworks.com/help/matlab/ref/diff.html**

**Trapezoidal Rule of Numerical Integration | Programming Numerical Methods in MATLAB - YouTube**

**Course: 60023055 - Análise Matemática II (isec.pt)**